

INDIAN INSTITUTE OF TECHNOLOGY MADRAS

Learning from Comparison Data (First draft)

A Project Report

Under the Supervision of

Prof. Suryanarayana Sankagiri

Submitted by

Nitesh Kumar Shah

ID25M806

Department of Data Science and Artificial Intelligence
Indian Institute of Technology Madras
Chennai – 600036

2026

Abstract

(Not Started yet)

Contents

Abstract **i**

Notation **vi**

1 Introduction **1**

1.1 Motivation 1

2 Background and Choice Models **2**

2.1 Choice Data and Choice Sets 2

2.2 Independence of Irrelevant Alternatives 2

2.2.1 Worked Example 3

2.3 Multinomial Logit Model 4

2.3.1 Shift Invariance and the Zero-Sum Constraint 4

3 From Multinomial Logit to Context-Dependent Choice Models **6**

3.1 Universal Logit Representation 6

3.2 Decomposition of Context-Dependent Utility 6

3.2.1 Statement of Lemma 1 6

3.2.2 Interpretation of the Different Orders 7

3.2.3 Worked Example 7

3.3 Hierarchy of Choice Models 8

3.3.1 The Model Classes $\mathcal{M}_p$ 8

3.3.2 Number of Free Parameters 9

3.3.3 MNL as the First-Order Model 9

3.4 Context-Dependent Random Utility Model 9

3.4.1 Second-Order Truncation 10

3.4.2 Reparameterisation of the Model 10

3.4.3 CDM Choice Probability 11

3.4.4 Interpretation of the Parameters 11

3.4.5 Number of Parameters 12

4 Bradley–Terry Model and Maximum Likelihood 13
4.1 Pairwise Comparison Setting . . . 13
4.2 Parameter Identifiability and Centring . . . 13
4.3 Maximum Likelihood Estimation . . . 14
4.4 Gradient-Based Estimation . . . 15
4.5 Numerical Experiments . . . 16
4.5.1 Baseline Parameter Recovery . . . 16
4.5.2 Effect of the Number of Comparisons . . . 18
4.6 Existence and Uniqueness of the MLE . . . 19
4.6.1 Ford's Condition . . . 19
4.6.2 Case Where a Finite MLE Exists . . . 19
4.6.3 Perfect Separation and Nonexistence of a Finite MLE . . . 20
4.7 Comparison of Optimization Methods . . . 21
4.7.1 Experimental Setup . . . 21
4.7.2 Final Solution Quality . . . 21
4.7.3 Convergence Criterion . . . 22
4.7.4 Handcrafted and PyTorch Runtime . . . 22

5 Experimental Study of the Full Context-Dependent Model 24
5.1 Experimental Setup . . . 24
5.2 CDM-to-CDM Parameter Recovery . . . 24
5.2.1 Baseline Recovery Experiment . . . 24
5.3 Recovery as the Number of Observations Increases . . . 25
5.3.1 Observation-Scaling Results . . . 25
5.4 Scaling with the Number of Items . . . 27
5.4.1 Fixed-Observation Results . . . 27
5.5 Learning CDM-Generated Choices with a BTL/Luce Model . . . 27
5.5.1 Objective . . . 27
5.5.2 Experimental Setup . . . 27
5.5.3 Baseline CDM-to-BTL Comparison . . . 27
5.5.4 Effect of the Number of Observations . . . 27
5.5.5 Effect of Context-Interaction Strength . . . 27
5.5.6 Key Observations . . . 27
5.5.7 Summary . . . 27

A Additional Derivations 29

B Reproducibility Details 30

List of Figures

4.1 Comparison of the true and estimated parameters for the baseline Bradley–Terry experiment with 10 items and 1000 pairwise comparisons. 17
4.2 Mean Bradley–Terry parameter-recovery RMSE as the number of comparisons increases. The error bars show one standard deviation across 20 independent runs. 18
4.3 Optimization behaviour when Ford's condition holds. 20
4.4 Optimization behaviour when Ford's condition fails. 20
4.5 Final gradient norm relative to the convergence tolerance. Values at or below one satisfy the convergence criterion. 22
4.6 Ratio of PyTorch runtime to handcrafted runtime for each optimization method. A value greater than one indicates that the PyTorch implementation required more time. 23

5.1 Recovery of the full CDM interaction matrix. 24
5.2 Full CDM recovery as the number of observations increases. 25
5.3 Full CDM recovery with a fixed number of observations as the number of items increases. 26
5.4 Predictive performance and test misspecification gap for BTL/Luce models fitted to CDM-generated choice data as the number of observations increases. Points represent means over ten independent runs, and error bars represent one standard deviation. 28
5.5 Effect of CDM interaction strength on the predictive performance and test misspecification gap of the fitted BTL/Luce model. Points represent means over ten independent runs, and error bars represent one standard deviation. 28

List of Tables

2.1 Alternatives and their assigned weights 3

4.1 Configuration and result of the baseline Bradley–Terry parameter-recovery experiment. 17
4.2 Parameter-recovery error for different numbers of pairwise comparisons. Each result is computed over 20 independent runs. 18
4.3 Comparison of the optimization methods. The runtime ratio is PyTorch runtime divided by handcrafted runtime. 21

5.1 Average negative log-likelihood in the baseline CDM-to-CDM recovery experiment. 24
5.2 Full CDM recovery as the number of observations increases. 25
5.3 Full CDM recovery with 50,000 observations as the number of items increases. 26
5.4 Test performance of BTL/Luce models fitted to CDM-generated data as the number of observations increases. 27
5.5 Effect of CDM interaction strength on BTL/Luce model misspecification. 28

Notation

Symbol	Meaning
$\mathcal{X}$	Universal set of alternatives
A, B, C	Choice sets containing available alternatives
x, y	Individual alternatives or items
$v(x)$	Item-level utility or score of alternative x
$u(x \mid C)$	Context-dependent utility of x in choice set C
θ_i	Bradley–Terry score of item i
$P(x \mid C)$	Probability of choosing x from C
$i \succ j$	Item i is preferred to or defeats item j
$L(\theta)$	Likelihood function
$\ell(\theta)$	Log-likelihood function
n	Number of alternatives in the universal set, $n = \lvert\mathcal{X}\rvert$
u_{xz}	Effect of context alternative z on target alternative x
U	Matrix containing the pairwise parameters u_{xz}

Chapter 1

Introduction

1.1 Motivation

Let us imagine that someone has to travel from IIT Madras to the airport. The person has four realistic choices: car, bike, taxi, and bus. Based on cost, comfort, and speed, the person may have a natural preference ranking among these alternatives. For example, the ranking may be

$$\text{Car} \succ \text{Taxi} \succ \text{Bike} \succ \text{Bus}. \tag{1.1}$$

Now suppose that a fifth alternative is added. This alternative is walking, which may take almost three hours and is therefore unlikely to be seriously considered. Common sense suggests that adding walking should not change the person's relative preference between car, taxi, bike, and bus. Walking is an irrelevant alternative in this situation. Therefore, the relative preference between the original alternatives should remain unchanged. This idea is called the *Independence of Irrelevant Alternatives* property. It is one of the central properties behind classical models of choice. However, human preferences may not always behave exactly as assumed by classical multinomial logit models. Experimental evidence indicates that adding or removing alternatives can change how attractive the existing alternatives appear. Such behaviour is described as a violation of the Independence of Irrelevant Alternatives property [1].

Chapter 2

Background and Choice Models

2.1 Choice Data and Choice Sets

Let $\mathcal{X}$ denote the universal set of alternatives. A choice set is a subset of $\mathcal{X}$ containing the alternatives available to a decision-maker at a particular time. For the commute-mode example, the universal set is

$$\mathcal{X} = \{\text{Car}, \text{Taxi}, \text{Bike}, \text{Bus}, \text{Walking}\}. \tag{2.1}$$

The walking alternative remains a member of the universal set even when it does not appear in a particular choice set. All the smaller choice sets are constructed from the same fixed universal set. Let $A, B \subseteq \mathcal{X}$ be two choice sets. Let x and y be alternatives belonging to the intersection $A \cap B$.

2.2 Independence of Irrelevant Alternatives

For a universal set $\mathcal{X}$ and choice sets $A, B \subseteq \mathcal{X}$, the Independence of Irrelevant Alternatives property for two alternatives x and y can be written as

$$\frac{P(x \mid A)}{P(y \mid A)} = \frac{P(x \mid B)}{P(y \mid B)}, \qquad x, y \in A \cap B. \tag{2.2}$$

The equation states that if x and y are present in both choice sets, then their relative odds should not change merely because other alternatives have been added or removed.

Table 2.1: Alternatives and their assigned weights

Item	Symbol	Weight w
Car	C	0.40
Taxi	T	0.25
Bike	Bi	0.15
Bus	Bu	0.12
Walking	W	0.08
Total		1.00

2.2.1 Worked Example

Consider the following two choice sets:

$$A = \{C, T, Bi\}, \qquad \sum_{x \in A} w_x = 0.80,$$
$$B = \{C, T, W\}, \qquad \sum_{x \in B} w_x = 0.73.$$

We want to verify whether

$$\frac{P(C \mid A)}{P(T \mid A)} = \frac{P(C \mid B)}{P(T \mid B)}. \tag{2.3}$$

For choice set A,

$$P(C \mid A) = \frac{w_C}{\sum_{x \in A} w_x} = \frac{0.40}{0.80} = 0.50,$$
$$P(T \mid A) = \frac{w_T}{\sum_{x \in A} w_x} = \frac{0.25}{0.80} = 0.3125.$$

Therefore,

$$\frac{P(C \mid A)}{P(T \mid A)} = \frac{0.40/0.80}{0.25/0.80} = \frac{0.40}{0.25} = 1.6. \tag{2.4}$$

For choice set B,

$$P(C \mid B) = \frac{w_C}{\sum_{x \in B} w_x} = \frac{0.40}{0.73},$$
$$P(T \mid B) = \frac{w_T}{\sum_{x \in B} w_x} = \frac{0.25}{0.73}.$$

Therefore,

$$\frac{P(C \mid B)}{P(T \mid B)} = \frac{0.40/0.73}{0.25/0.73} = \frac{0.40}{0.25} = 1.6. \tag{2.5}$$

Thus,

$$\frac{P(C \mid A)}{P(T \mid A)} = \frac{P(C \mid B)}{P(T \mid B)}. \tag{2.6}$$

Hence, the Independence of Irrelevant Alternatives property holds in this worked example.

2.3 Multinomial Logit Model

The classical multinomial logit model assigns a score $v(x)$ to every alternative x. The probability of selecting x from choice set C is

$$P(x \mid C) = \frac{\exp(v(x))}{\sum_{y \in C} \exp(v(y))}. \tag{2.7}$$

The scores are shift-invariant. A common identification constraint is

$$\sum_{x \in \mathcal{X}} v(x) = 0. \tag{2.8}$$

2.3.1 Shift Invariance and the Zero-Sum Constraint

Consider two alternatives, car and taxi, with scores

$$\begin{aligned} v(C) &= 2.5, \\ v(T) &= -0.9. \end{aligned}$$

Their exponentiated scores are

$$\begin{aligned} \exp(v(C)) &= \exp(2.5) \approx 12.182, \\ \exp(v(T)) &= \exp(-0.9) \approx 0.407. \end{aligned}$$

Therefore, the probability of choosing car from the set $\{C, T\}$ is

$$\begin{aligned} P(C \mid \{C, T\}) &= \frac{\exp(2.5)}{\exp(2.5) + \exp(-0.9)} \\ &= \frac{12.182}{12.182 + 0.407} \\ &\approx 0.968. \end{aligned}$$

However,

$$v(C) + v(T) = 2.5 - 0.9 = 1.6 \neq 0. \tag{2.9}$$

Suppose the same constant k is added to both scores. To satisfy the zero-sum constraint, choose k such that

$$\begin{aligned}(2.5 + k) + (-0.9 + k) &= 0,\\ 1.6 + 2k &= 0,\\ k &= -0.8.\end{aligned}$$

After applying this shift,

$$\begin{aligned}v(C) &= 2.5 - 0.8 = 1.7,\\ v(T) &= -0.9 - 0.8 = -1.7.\end{aligned}$$

The probability obtained using these centred scores is

$$\begin{aligned}P(C \mid \{C, T\}) &= \frac{\exp(1.7)}{\exp(1.7) + \exp(-1.7)}\\ &= \frac{5.474}{5.474 + 0.183}\\ &\approx 0.968.\end{aligned}$$

Thus, adding the same constant to every score does not change the choice probabilities. The zero-sum constraint selects one representative from all score vectors that produce the same probabilities.

Chapter 3

From Multinomial Logit to Context-Dependent Choice Models

3.1 Universal Logit Representation

In a universal logit model, the utility of an alternative can depend on the choice set in which it appears. Instead of assigning only one fixed utility to an alternative, the model allows it to have different utilities in different contexts. The probability of selecting x from C is

$$P(x \mid C) = \frac{\exp(u(x \mid C))}{\sum_{y \in C} \exp(u(y \mid C))}. \tag{3.1}$$

An identification constraint can be imposed separately for each choice set:

$$\sum_{y \in C} u(y \mid C) = 0, \qquad \text{for every } C \subseteq \mathcal{X}. \tag{3.2}$$

3.2 Decomposition of Context-Dependent Utility

3.2.1 Statement of Lemma 1

Lemma 3.1 (Decomposition of context-dependent utility)**.** *For a universal logit model, the context-dependent utility $u(x \mid C)$ can be written as a sum of terms indexed by subsets B of $C \setminus \{x\}$ [1]:*

$$u(x \mid C) = \sum_{B \subseteq C \setminus \{x\}} v(x \mid B). \tag{3.3}$$

Equivalently,

$$u(x \mid C) = v(x) + \sum_{y \in C \setminus \{x\}} v(x \mid \{y\}) + \sum_{\{y,z\} \subseteq C \setminus \{x\}} v(x \mid \{y,z\}) + \cdots . \tag{3.4}$$

The parameters satisfy

$$\sum_{x \notin B} v(x \mid B) = 0 \qquad \textit{for every } B \subseteq \mathcal{X}. \tag{3.5}$$

3.2.2 Interpretation of the Different Orders

The term $v(x)$ represents the intrinsic preference associated with alternative x. It does not depend on the other alternatives present in the choice set. The term $v(x \mid \{z\})$ represents the contextual contribution of alternative z to the utility of alternative x. Similarly, a term such as $v(x \mid \{y,z\})$ represents a higher-order interaction involving the joint presence of alternatives y and z.

3.2.3 Worked Example

Consider the choice set

$$A = \{C, T, Bi\}. \tag{3.6}$$

The subsets of A are

$$\emptyset,\ \{C\},\ \{T\},\ \{Bi\},\ \{C,T\},\ \{T,Bi\},\ \{Bi,C\},\ \{C,T,Bi\}. \tag{3.7}$$

The utility expansions are

$$\begin{aligned} u(C \mid A) &= v(C) + v(C \mid \{T\}) + v(C \mid \{Bi\}) + v(C \mid \{T,Bi\}), \\ u(T \mid A) &= v(T) + v(T \mid \{C\}) + v(T \mid \{Bi\}) + v(T \mid \{C,Bi\}), \\ u(Bi \mid A) &= v(Bi) + v(Bi \mid \{C\}) + v(Bi \mid \{T\}) + v(Bi \mid \{C,T\}). \end{aligned}$$

The corresponding constraints are

$$\begin{aligned} v(C) + v(T) + v(Bi) &= 0, \\ v(T \mid \{C\}) + v(Bi \mid \{C\}) &= 0, \\ v(C \mid \{T\}) + v(Bi \mid \{T\}) &= 0, \\ v(C \mid \{Bi\}) + v(T \mid \{Bi\}) &= 0, \\ v(C \mid \{T, Bi\}) &= 0, \\ v(T \mid \{C, Bi\}) &= 0, \\ v(Bi \mid \{C, T\}) &= 0. \end{aligned}$$

The final three constraints force the highest-order terms to zero. Therefore, the utilities reduce to

$$\begin{aligned} u(C \mid A) &= v(C) + v(C \mid \{T\}) + v(C \mid \{Bi\}), \\ u(T \mid A) &= v(T) + v(T \mid \{C\}) + v(T \mid \{Bi\}), \\ u(Bi \mid A) &= v(Bi) + v(Bi \mid \{C\}) + v(Bi \mid \{T\}). \end{aligned}$$

There are nine remaining parameters and four independent constraints. Hence, the number of free parameters is

$$9 - 4 = 5. \tag{3.8}$$

3.3 Hierarchy of Choice Models

The decomposition gives a hierarchy of choice models. A lower-order model retains simpler contextual interactions, while a higher-order model allows the utility of an alternative to depend on larger subsets of the choice set. Increasing the order makes the model more flexible, but also increases the number of parameters that must be estimated.

3.3.1 The Model Classes $\mathcal{M}_p$

Let $\mathcal{M}_p$ denote the model obtained by retaining utility terms up to order p and setting all higher-order terms equal to zero. This produces the nested hierarchy

$$\mathcal{M}_1 \subseteq \mathcal{M}_2 \subseteq \cdots \subseteq \mathcal{M}_{n-1}. \tag{3.9}$$

The first-order model retains only $v(x)$. The second-order model additionally retains terms of the form $v(x \mid \{z\})$. Higher-order models include terms involving larger subsets of alternatives.

3.3.2 Number of Free Parameters

For the second-order model, there are n intrinsic parameters of the form $v(x)$ and $n(n-1)$ contextual parameters of the form $v(x \mid \{z\})$, where $x \neq z$. Thus, before applying the constraints, the number of parameters is

$$n + n(n-1) = n^2. \tag{3.10}$$

The intrinsic parameters satisfy one constraint:

$$\sum_{x \in \mathcal{X}} v(x) = 0. \tag{3.11}$$

For every $z \in \mathcal{X}$, the contextual parameters satisfy

$$\sum_{x \in \mathcal{X} \setminus \{z\}} v(x \mid \{z\}) = 0. \tag{3.12}$$

There are n contextual constraints. Therefore, the total number of constraints is $n+1$, and the number of free parameters is

$$n^2 - (n+1) = n^2 - n - 1. \tag{3.13}$$

3.3.3 MNL as the First-Order Model

The multinomial logit model is the first-order member of this hierarchy. In $\mathcal{M}_1$, all contextual terms are set to zero, leaving

$$u(x \mid C) = v(x). \tag{3.14}$$

Substituting this expression into the universal logit representation gives

$$P(x \mid C) = \frac{\exp(v(x))}{\sum_{y \in C} \exp(v(y))}, \tag{3.15}$$

which is the multinomial logit model.

3.4 Context-Dependent Random Utility Model

The first-order model does not allow the utility of an alternative to change when other alternatives are added to or removed from the choice set. To represent such context effects without retaining every higher-order interaction, we consider the second-order

model $\mathcal{M}_2$.

3.4.1 Second-Order Truncation

Starting from the universal logit decomposition,

$$\begin{aligned} u(x \mid C) = v(x) &+ \sum_{z \in C \setminus \{x\}} v(x \mid \{z\}) \\ &+ \sum_{\{y,z\} \subseteq C \setminus \{x\}} v(x \mid \{y,z\}) + \cdots , \end{aligned} \tag{3.16}$$

the second-order model sets all terms involving two or more contextual alternatives equal to zero. Hence,

$$u(x \mid C) = v(x) + \sum_{z \in C \setminus \{x\}} v(x \mid \{z\}). \tag{3.17}$$

The corresponding choice probability is

$$P(x \mid C) = \frac{\exp\left(v(x) + \sum_{z \in C \setminus \{x\}} v(x \mid \{z\}) \right)}{\sum_{y \in C} \exp\left(v(y) + \sum_{z \in C \setminus \{y\}} v(y \mid \{z\}) \right)}. \tag{3.18}$$

Here, $v(x)$ represents the intrinsic preference associated with x, while $v(x \mid \{z\})$ represents the contribution of context alternative z to the utility of x.

3.4.2 Reparameterisation of the Model

The second-order formulation contains two types of parameters: the intrinsic parameters $v(x)$ and the contextual parameters $v(x \mid \{z\})$. These can be combined into a single set of pairwise parameters. For $x \neq z$, define

$$u_{xz} = v(x \mid \{z\}) - v(z). \tag{3.19}$$

Here, x is the target alternative and z is the context alternative. Using this definition, the reparameterised utility is

$$\begin{aligned}\widetilde{u}(x \mid C) &= \sum_{z \in C \setminus \{x\}} u_{xz} \\ &= \sum_{z \in C \setminus \{x\}} (v(x \mid \{z\}) - v(z)) \\ &= v(x) + \sum_{z \in C \setminus \{x\}} v(x \mid \{z\}) - \sum_{z \in C} v(z) \\ &= u(x \mid C) - \sum_{z \in C} v(z).\end{aligned}$$

The term $\sum_{z \in C} v(z)$ depends only on the choice set C and is the same for every candidate in that choice set. Subtracting the same quantity from every candidate utility does not change the softmax probabilities:

$$\frac{\exp(q_x - a)}{\sum_{y \in C} \exp(q_y - a)} = \frac{\exp(q_x)}{\sum_{y \in C} \exp(q_y)}. \tag{3.20}$$

Therefore, reparameterisation changes the form of the parameters without changing the choice probabilities.

3.4.3 CDM Choice Probability

After reparameterisation, the utility of alternative x in choice set C is

$$\widetilde{u}(x \mid C) = \sum_{z \in C \setminus \{x\}} u_{xz}. \tag{3.21}$$

Therefore, the CDM choice probability is

$$P(x \mid C) = \frac{\exp\left(\sum_{z \in C \setminus \{x\}} u_{xz}\right)}{\sum_{y \in C} \exp\left(\sum_{z \in C \setminus \{y\}} u_{yz}\right)}. \tag{3.22}$$

3.4.4 Interpretation of the Parameters

The parameter u_{xz} represents the effect of context alternative z on target alternative x. For example, $u_{\text{Car,Bus}}$ represents the contextual effect of Bus on the utility associated with Car. In general, u_{xz} and u_{zx} can be different because they represent effects in opposite directions.

3.4.5 Number of Parameters

The reparameterised parameters can be arranged in a matrix $U \in \mathbb{R}^{n \times n}$. The diagonal entries are set to zero because an alternative is not treated as its own context:

$$u_{xx} = 0. \tag{3.23}$$

Therefore, U contains

$$n^2 - n = n(n-1) \tag{3.24}$$

off-diagonal parameters. These parameters satisfy the identification constraint

$$\sum_{x \in \mathcal{X}} \sum_{z \in \mathcal{X} \setminus \{x\}} u_{xz} = 0. \tag{3.25}$$

Hence, the number of free parameters is

$$n(n-1) - 1 = n^2 - n - 1. \tag{3.26}$$

This agrees with the count obtained from the original second-order formulation in eq. (3.13). For example, suppose $\mathcal{X} = \{A, B, C\}$. Then

$$U = \begin{pmatrix} 0 & u_{AB} & u_{AC} \\ u_{BA} & 0 & u_{BC} \\ u_{CA} & u_{CB} & 0 \end{pmatrix}. \tag{3.27}$$

The matrix has six off-diagonal parameters. After applying the constraint

$$u_{AB} + u_{AC} + u_{BA} + u_{BC} + u_{CA} + u_{CB} = 0, \tag{3.28}$$

the number of free parameters is

$$6 - 1 = 5. \tag{3.29}$$

This is consistent with the five free parameters obtained in eq. (3.8).

Chapter 4

Bradley–Terry Model and Maximum Likelihood

4.1 Pairwise Comparison Setting

To understand the underlying mechanics of a model for choice data, we consider observed pairwise comparisons and assume a probabilistic model. The Bradley–Terry model is used for this purpose. The goal is to estimate unknown parameters that provide a compressed representation of the observed comparison data. For items i and j, with parameters θ_i and θ_j, the Bradley–Terry model defines

$$P(i \succ j) = \frac{\exp(\theta_i)}{\exp(\theta_i) + \exp(\theta_j)}. \tag{4.1}$$

Dividing the numerator and denominator by $\exp(\theta_i)$ gives

$$P(i \succ j) = \frac{1}{1 + \exp\left(-(\theta_i - \theta_j)\right)}. \tag{4.2}$$

Therefore, the probability depends only on the parameter difference $\theta_i - \theta_j$.

4.2 Parameter Identifiability and Centring

The Bradley–Terry probability depends only on the difference between two parameters. Consequently, adding the same constant c to every parameter does not change any pairwise comparison probability:

$$\begin{aligned} P(i \succ j) &= \frac{\exp(\theta_i + c)}{\exp(\theta_i + c) + \exp(\theta_j + c)} \\ &= \frac{\exp(\theta_i)}{\exp(\theta_i) + \exp(\theta_j)}. \end{aligned}$$

Therefore, the parameter vectors

$$(\theta_1, \ldots, \theta_n) \qquad \text{and} \qquad (\theta_1 + c, \ldots, \theta_n + c)$$

represent the same Bradley–Terry model. The individual parameter values are consequently not identifiable without an additional constraint.

We remove this additive ambiguity using the zero-sum constraint

$$\sum_{i=1}^{n} \theta_i = 0. \tag{4.3}$$

For any parameter vector $\boldsymbol{\theta}$, a centred version can be obtained by subtracting its mean:

$$\theta_i^{\text{centred}} = \theta_i - \frac{1}{n} \sum_{j=1}^{n} \theta_j. \tag{4.4}$$

This transformation preserves every parameter difference and therefore does not change the pairwise comparison probabilities. In the numerical experiments, both the true and estimated parameter vectors are centred before their recovery error is calculated.

4.3 Maximum Likelihood Estimation

Suppose that the observed dataset contains m pairwise comparisons:

$$\mathcal{D} = \{(w_t, l_t)\}_{t=1}^{m}, \tag{4.5}$$

where w_t and l_t denote the winner and loser of comparison t, respectively.

Conditional on the compared pairs, the observed comparison outcomes are modelled as independent. The probability of the observed outcome in comparison t is

$$p_t = P(w_t \succ l_t) = \frac{\exp(\theta_{w_t})}{\exp(\theta_{w_t}) + \exp(\theta_{l_t})}. \tag{4.6}$$

The likelihood of the complete dataset is therefore

$$L(\boldsymbol{\theta}; \mathcal{D}) = \prod_{t=1}^{m} \frac{\exp(\theta_{w_t})}{\exp(\theta_{w_t}) + \exp(\theta_{l_t})}. \tag{4.7}$$

Taking the logarithm gives the log-likelihood

$$\begin{aligned}\ell(\boldsymbol{\theta};\mathcal{D}) &= \sum_{t=1}^{m} \log\left(\frac{\exp(\theta_{w_t})}{\exp(\theta_{w_t})+\exp(\theta_{l_t})}\right) \\ &= \sum_{t=1}^{m} \left[\theta_{w_t} - \log\left(\exp(\theta_{w_t})+\exp(\theta_{l_t})\right)\right].\end{aligned}$$

Maximizing the log-likelihood is equivalent to minimizing its negative. In the implementation, we use the average negative log-likelihood

$$\mathcal{L}(\boldsymbol{\theta}) = -\frac{1}{m}\ell(\boldsymbol{\theta};\mathcal{D}) \tag{4.8}$$

$$= \frac{1}{m}\sum_{t=1}^{m} \log\left(1+\exp(\theta_{l_t}-\theta_{w_t})\right). \tag{4.9}$$

Using the average rather than the total negative log-likelihood makes the reported loss values comparable across datasets containing different numbers of comparisons.

The maximum likelihood estimate is written as

$$\widehat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}), \qquad \text{subject to} \qquad \sum_{i=1}^{n} \theta_i = 0. \tag{4.10}$$

4.4 Gradient-Based Estimation

The average negative log-likelihood in eq. (4.9) is minimized using gradient descent. For one observed winner–loser pair (w_t, l_t), the loss contribution is

$$\mathcal{L}_t = \log\left(1+\exp(\theta_{l_t}-\theta_{w_t})\right). \tag{4.11}$$

Let

$$p_t = P(w_t \succ l_t). \tag{4.12}$$

The derivative of this loss with respect to the winner parameter is

$$\frac{\partial \mathcal{L}_t}{\partial \theta_{w_t}} = p_t - 1, \tag{4.13}$$

while the derivative with respect to the loser parameter is

$$\frac{\partial \mathcal{L}_t}{\partial \theta_{l_t}} = 1 - p_t. \tag{4.14}$$

Therefore, when the observed winner is assigned a small winning probability, the magnitude of the update is larger. When the model already assigns the observed winner a probability close to one, the magnitude of the update is smaller.

The gradient of the average negative log-likelihood with respect to an arbitrary parameter θ_k is

$$\frac{\partial \mathcal{L}}{\partial \theta_k} = \frac{1}{m} \sum_{t=1}^{m} \left[(p_t - 1)\mathbb{I}\{k = w_t\} + (1 - p_t)\mathbb{I}\{k = l_t\} \right], \tag{4.15}$$

where $\mathbb{I}\{\cdot\}$ is the indicator function.

At gradient-descent iteration s, the complete parameter vector is updated according to

$$\boldsymbol{\theta}^{(s+1)} = \boldsymbol{\theta}^{(s)} - \eta \nabla \mathcal{L} \left(\boldsymbol{\theta}^{(s)} \right), \tag{4.16}$$

where $\eta > 0$ is the learning rate.

After an update, the parameter vector is centred using

$$\theta_i^{(s+1)} \leftarrow \theta_i^{(s+1)} - \frac{1}{n} \sum_{j=1}^{n} \theta_j^{(s+1)}. \tag{4.17}$$

This centring step enforces the zero-sum constraint while preserving all pairwise parameter differences. The numerical experiments use this gradient-based procedure to estimate the Bradley–Terry parameters from the simulated winner–loser observations.

4.5 Numerical Experiments

4.5.1 Baseline Parameter Recovery

The first experiment checks whether the parameters used to generate simulated Bradley–Terry comparisons can be recovered from the observed data.

We consider $n = 10$ items. The true parameter vector is constructed using equally spaced values between -1.5 and 1.5. The vector is centred so that

$$\sum_{i=1}^{n} \theta_i = 0. \tag{4.18}$$

A total of $m = 1000$ pairwise comparisons are generated. Each pair is sampled uniformly, and the winner is sampled according to the Bradley–Terry probability in eq. (4.1). The random seed is fixed at 20260831 to make the experiment reproducible.

The unknown parameters are estimated by minimizing the average negative log-likelihood using gradient descent. The learning rate is set to 0.1, and the optimizer is run for a maximum of 1000 iterations.

The parameter-recovery error is measured using the root mean squared error

$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}\left(\widehat{\theta}_i - \theta_i\right)^2}, \tag{4.19}$$

where θ_i is the true parameter and $\widehat{\theta}_i$ is its estimate.

Table 4.1: Configuration and result of the baseline Bradley–Terry parameter-recovery experiment.

Quantity	Value
Number of items, n	10
Number of comparisons, m	1000
Random seed	20260831
Learning rate	0.1
Maximum iterations	1000
Parameter-recovery RMSE	0.1224

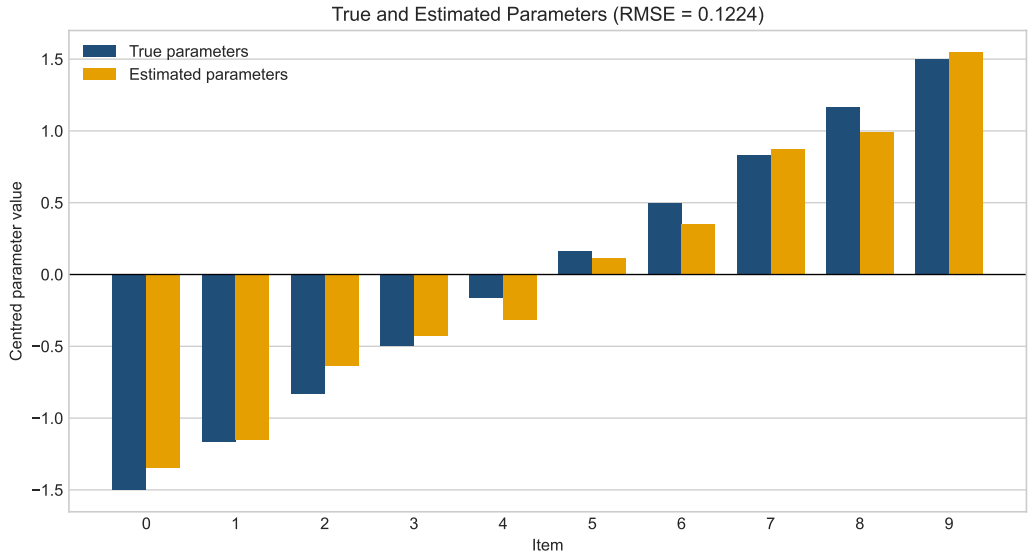


Figure 4.1: Comparison of the true and estimated parameters for the baseline Bradley–Terry experiment with 10 items and 1000 pairwise comparisons.

As shown in fig. 4.1, the estimated parameters follow the ordering and approximate magnitudes of the true parameters. The parameter-recovery RMSE for this simulated dataset is 0.1224.

This result provides a baseline example of parameter recovery. Since it is obtained from a single simulated dataset, it does not describe the variation across independent datasets. Repeated experiments are used in the next subsection to study how the recovery error changes with the number of observed comparisons.

4.5.2 Effect of the Number of Comparisons

The second experiment studies how the number of observed pairwise comparisons affects parameter recovery. The number of items is fixed at $n = 10$, and the same centred true parameter vector used in the baseline experiment is retained.

The number of comparisons is varied over

$$m \in \{50, 100, 200, 500, 1000, 2000, 5000\}. \tag{4.20}$$

For each value of m, the experiment is repeated independently 20 times. A new comparison dataset is generated in every run, while the true parameters, pair-sampling method, learning rate, and maximum number of gradient-descent iterations are kept fixed. The mean parameter-recovery RMSE and its standard deviation are then computed over the 20 runs.

Table 4.2: Parameter-recovery error for different numbers of pairwise comparisons. Each result is computed over 20 independent runs.

Number of comparisons	Mean RMSE	Standard deviation
50	0.7225	0.2172
100	0.4600	0.1247
200	0.3213	0.0819
500	0.1786	0.0487
1000	0.1412	0.0388
2000	0.1095	0.0309
5000	0.0780	0.0203

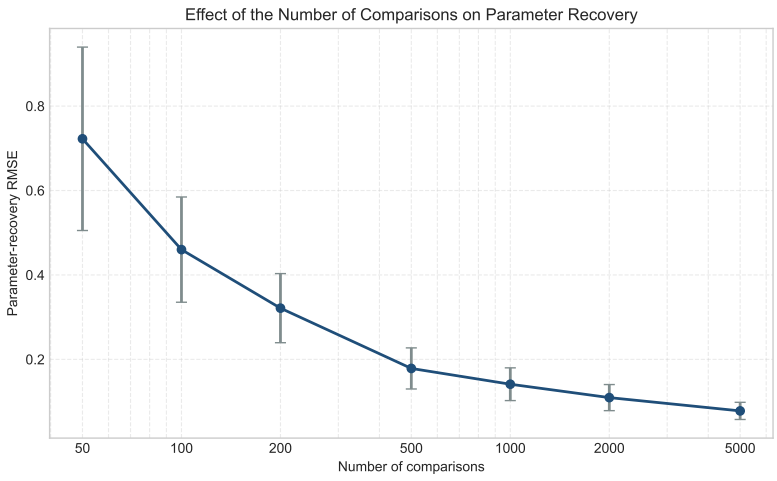


Figure 4.2: Mean Bradley–Terry parameter-recovery RMSE as the number of comparisons increases. The error bars show one standard deviation across 20 independent runs.

As shown in table 4.2 and fig. 4.2, the mean parameter-recovery RMSE decreases

as the number of observed comparisons increases. It decreases from 0.7225 with 50 comparisons to 0.0780 with 5000 comparisons.

The standard deviation also decreases from 0.2172 to 0.0203. Thus, in this experimental setting, increasing the amount of observed comparison data improves both the average recovery accuracy and the consistency of the estimates across independent datasets.

4.6 Existence and Uniqueness of the MLE

4.6.1 Ford's Condition

For the Bradley–Terry model, the existence of a finite maximum likelihood estimate depends on the observed comparison outcomes. These outcomes can be represented using a directed win graph, in which each vertex represents an item and an edge $i \to j$ indicates that item i has defeated item j.

Ford's condition holds when this directed win graph is strongly connected. Equivalently, for every nonempty proper subset of items, there must be at least one observed win from an item inside the subset against an item outside it, and vice versa.

When Ford's condition holds, the Bradley–Terry maximum likelihood estimate is finite. After imposing the centring constraint

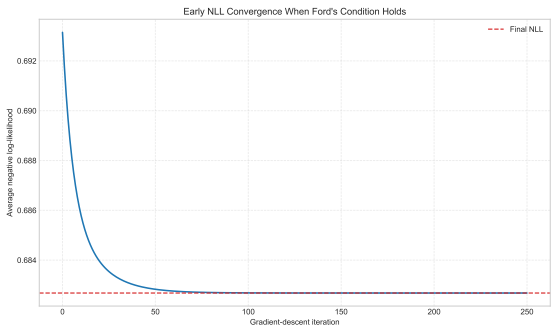
$$\sum_{i=1}^{n} \theta_i = 0,$$

the estimate is unique.

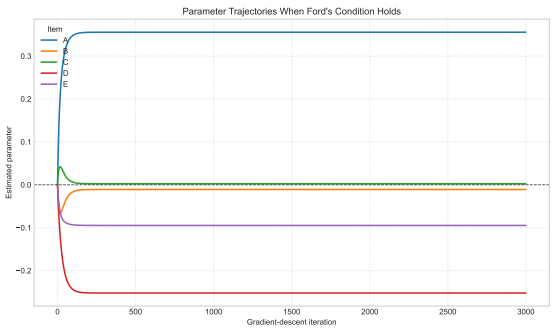
4.6.2 Case Where a Finite MLE Exists

In the first experiment, the comparison outcomes are constructed so that the directed win graph is strongly connected. Consequently, Ford's condition holds.

Figure 4.3 shows the optimization behaviour. The negative log-likelihood converges to a finite value, while the centred parameter estimates stabilize. This behaviour is consistent with the existence of a finite Bradley–Terry MLE.



(a) Negative log-likelihood during optimization.



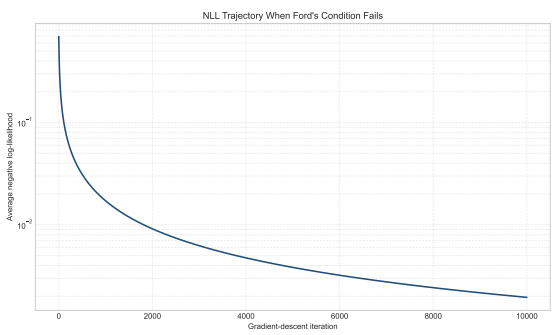
(b) Trajectories of the centred parameters.

Figure 4.3: Optimization behaviour when Ford's condition holds.

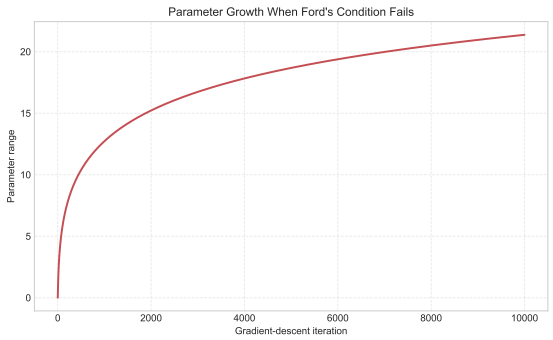
4.6.3 Perfect Separation and Nonexistence of a Finite MLE

In the second experiment, the observed outcomes follow a perfectly separated ordering. Although the corresponding undirected comparison graph is connected, the directed win graph is not strongly connected. Therefore, Ford's condition fails.

As shown in Figure 4.4, the negative log-likelihood continues to decrease, while the range of the estimated parameters continues to increase. Thus, the likelihood can be improved by increasing the parameter differences, but no finite parameter vector attains the maximum likelihood value.



(a) Negative log-likelihood under perfect separation.



(b) Growth of the parameter range.

Figure 4.4: Optimization behaviour when Ford's condition fails.

These experiments illustrate that ordinary connectivity of the comparison graph is not sufficient for the existence of a finite MLE. The direction of the observed wins, and hence strong connectivity of the directed win graph, is essential.

4.7 Comparison of Optimization Methods

4.7.1 Experimental Setup

The Bradley–Terry negative log-likelihood was minimized using five optimization methods: full-batch gradient descent, stochastic gradient descent (SGD), mini-batch gradient descent, gradient descent with momentum, and Adam.

Each method was implemented in two ways: a handcrafted implementation and an implementation using PyTorch. The same simulated comparison dataset and initial parameter vector were used for all methods.

A maximum budget of 10,000 epochs was used. Convergence was measured using the norm of the gradient evaluated on the complete dataset. The convergence tolerance was set to

$$\|\nabla \mathcal{L}(\boldsymbol{\theta})\|_2 \leq 10^{-4}. \tag{4.21}$$

For easier comparison, define the convergence ratio

$$r = \frac{\|\nabla \mathcal{L}(\boldsymbol{\theta})\|_2}{10^{-4}}. \tag{4.22}$$

Thus, a run satisfies the convergence criterion when $r \leq 1$.

4.7.2 Final Solution Quality

Table 4.3 summarizes the parameter-recovery error, convergence ratio, and relative runtime. Here, H and P denote the handcrafted and PyTorch implementations, respectively.

Table 4.3: Comparison of the optimization methods. The runtime ratio is PyTorch runtime divided by handcrafted runtime.

Optimizer	RMSE (H/P)	r (H/P)	Runtime ratio (P/H)
Full-batch GD	0.0381/0.0381	1.00/1.00	1.34
SGD	0.0393/0.0395	4.80/4.54	2.59
Mini-batch GD	0.0384/0.0387	0.90/6.31	27.82
Momentum GD	0.0404/0.0360	7.92/9.82	2.15
Adam	0.0408/0.0438	24.94/29.12	2.31

The recovery RMSE values are similar across the different methods, lying between 0.0360 and 0.0438. Therefore, all methods produced parameter estimates close to the true parameters in this experiment. However, similar recovery errors do not imply that every method satisfied the numerical convergence criterion.

4.7.3 Convergence Criterion

Figure 4.5 shows the final gradient norm relative to the selected tolerance.

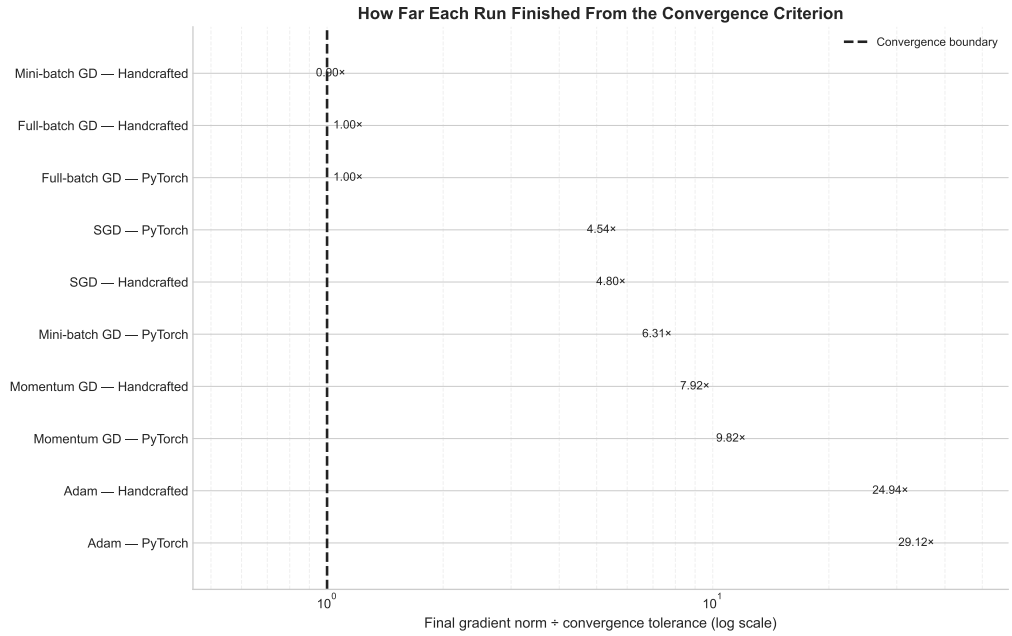


Figure 4.5: Final gradient norm relative to the convergence tolerance. Values at or below one satisfy the convergence criterion.

Both implementations of full-batch gradient descent satisfied the convergence criterion. The handcrafted mini-batch implementation also satisfied the criterion. The remaining methods did not reach the specified tolerance within the fixed epoch budget.

In particular, stochastic methods continue to fluctuate near a solution because their updates are computed using individual observations or mini-batches. Consequently, a small recovery error may still be obtained even when the full-gradient norm remains above the strict convergence tolerance.

4.7.4 Handcrafted and PyTorch Runtime

Figure 4.6 compares the runtime of the PyTorch implementation with its corresponding handcrafted implementation.

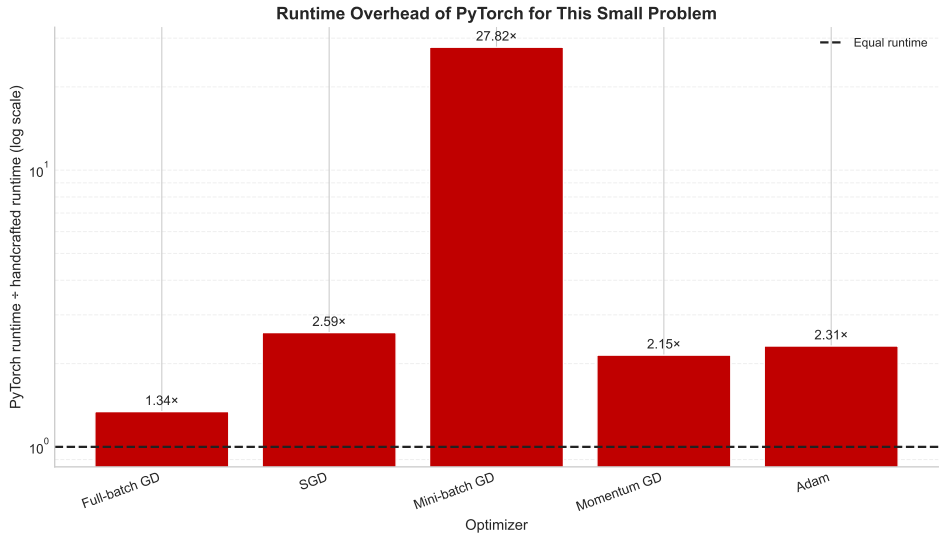


Figure 4.6: Ratio of PyTorch runtime to handcrafted runtime for each optimization method. A value greater than one indicates that the PyTorch implementation required more time.

For this small Bradley–Terry problem, the handcrafted implementations were faster than their PyTorch counterparts. The additional runtime is consistent with the overhead introduced by tensor operations and automatic differentiation on a small CPU-based problem.

This observation is specific to the present implementation and problem size. It should not be interpreted as a general comparison of PyTorch and handcrafted optimization, since PyTorch may become advantageous for larger models, larger datasets, or GPU-based computation.

Chapter 5

Experimental Study of the Full Context-Dependent Model

5.1 Experimental Setup

5.2 CDM-to-CDM Parameter Recovery

5.2.1 Baseline Recovery Experiment

Table 5.1: Average negative log-likelihood in the baseline CDM-to-CDM recovery experiment.

Evaluation	Average NLL
Uniform baseline—training	0.9015
True parameters—training	0.8275
Estimated parameters—training	0.8169
True parameters—test	0.8020
Estimated parameters—test	0.8190

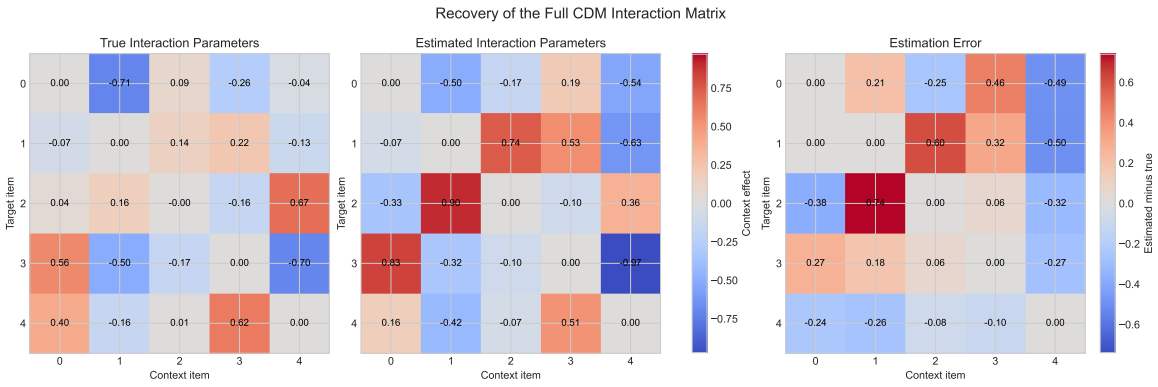


Figure 5.1: Recovery of the full CDM interaction matrix.

5.3 Recovery as the Number of Observations Increases

5.3.1 Observation-Scaling Results

Table 5.2: Full CDM recovery as the number of observations increases.

Observations	Runs	Mean RMSE	RMSE standard deviation	Mean correlation	Mean excess test NLL
1,000	10	0.3243	0.0888	0.7519	0.0100
5,000	10	0.1216	0.0321	0.9493	0.0023
10,000	10	0.0984	0.0247	0.9673	0.0016
50,000	10	0.0482	0.0115	0.9920	0.0003

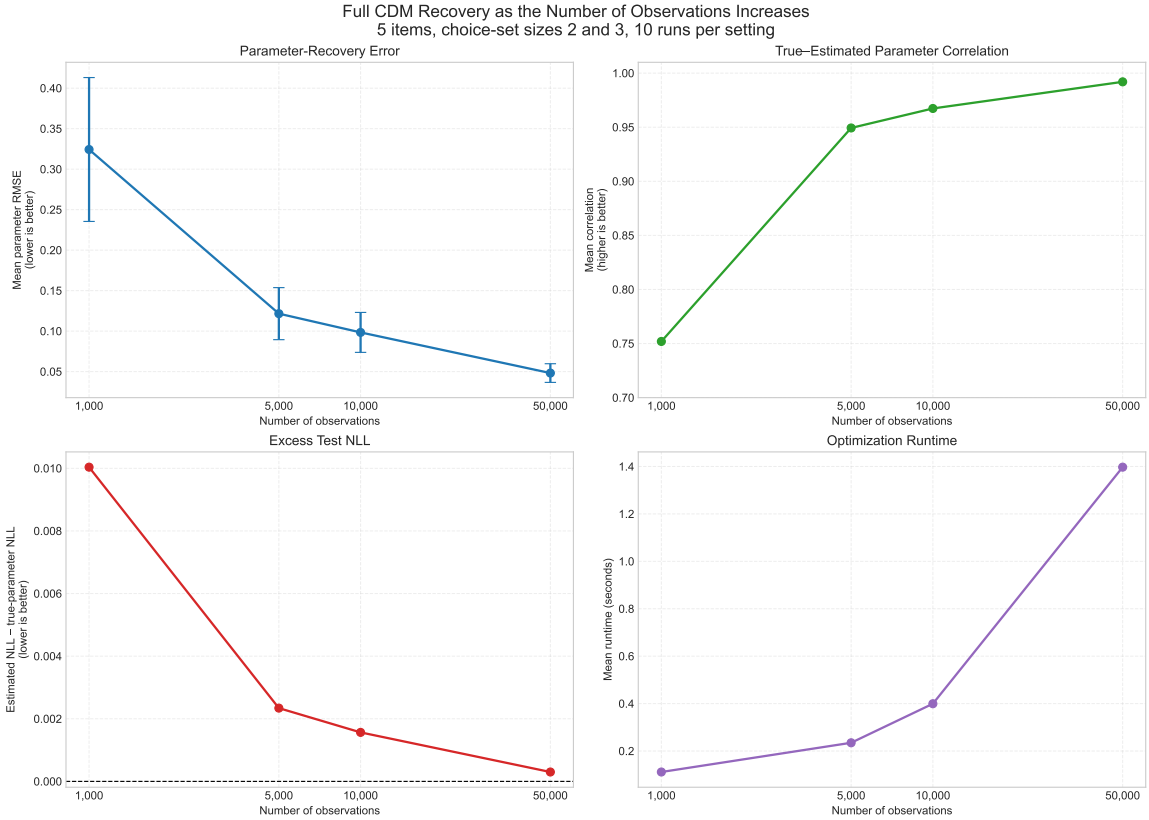


Figure 5.2: Full CDM recovery as the number of observations increases.

Table 5.3: Full CDM recovery with 50,000 observations as the number of items increases.

Items	Free parameters	Runs	Mean RMSE	RMSE standard deviation	Mean correlation	Mean excess test NLL
5	19	5	0.0520	0.0132	0.9843	0.0000
10	89	5	0.0945	0.0114	0.9692	0.0009
20	379	5	0.1683	0.0054	0.9192	0.0052
40	1,559	5	0.3303	0.0118	0.7860	0.0190
80	6,319	5	0.8017	0.0210	0.5099	0.1152

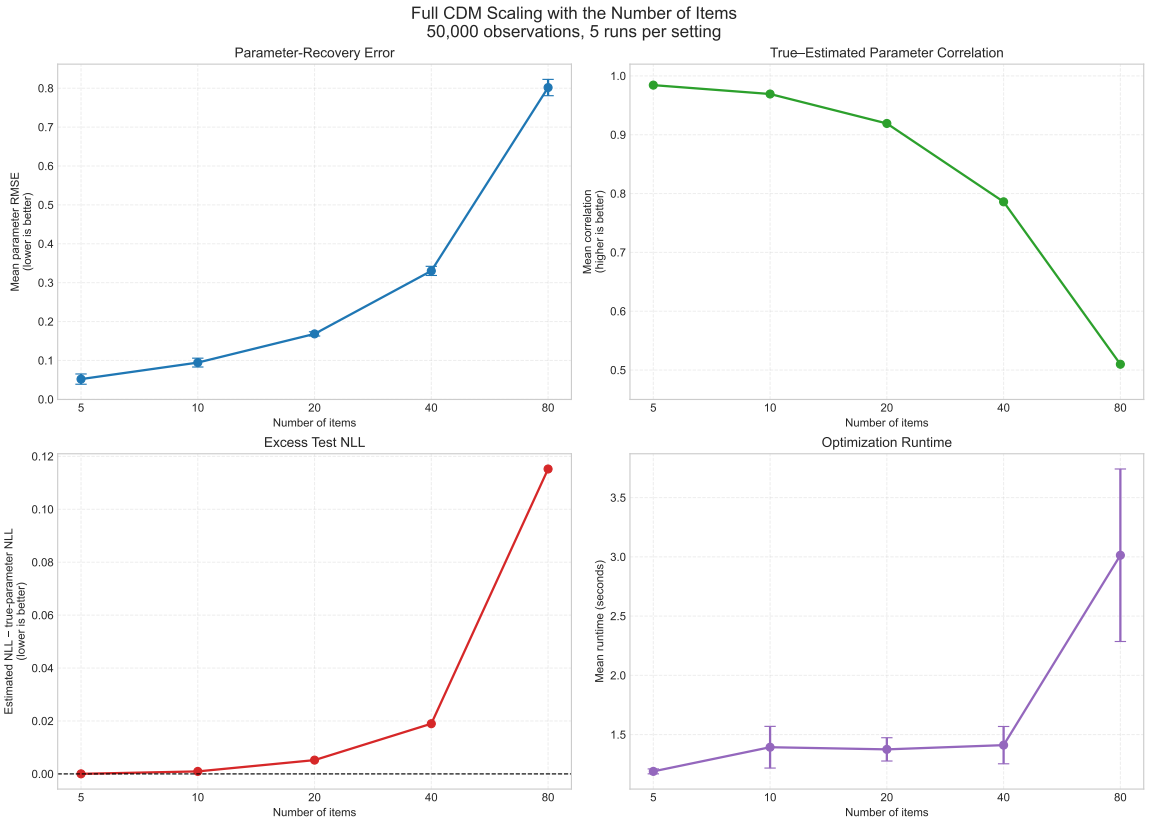


Figure 5.3: Full CDM recovery with a fixed number of observations as the number of items increases.

5.4 Scaling with the Number of Items

5.4.1 Fixed-Observation Results

5.5 Learning CDM-Generated Choices with a BTL/Luce Model

5.5.1 Objective

5.5.2 Experimental Setup

5.5.3 Baseline CDM-to-BTL Comparison

5.5.4 Effect of the Number of Observations

Table 5.4: Test performance of BTL/Luce models fitted to CDM-generated data as the number of observations increases.

Items	Observations	True CDM NLL	BTL/Luce NLL	Gap	Gap SD
5	1000	0.8065	0.8591	0.0526	0.0212
5	5000	0.8000	0.8535	0.0535	0.0104
5	10000	0.7907	0.8424	0.0517	0.0086
5	50000	0.7914	0.8456	0.0542	0.0030
10	1000	0.8484	0.8985	0.0502	0.0231
10	5000	0.8274	0.8837	0.0563	0.0124
10	10000	0.8207	0.8845	0.0638	0.0081
10	50000	0.8226	0.8823	0.0597	0.0034

5.5.5 Effect of Context-Interaction Strength

5.5.6 Key Observations

-
-
-
-

5.5.7 Summary

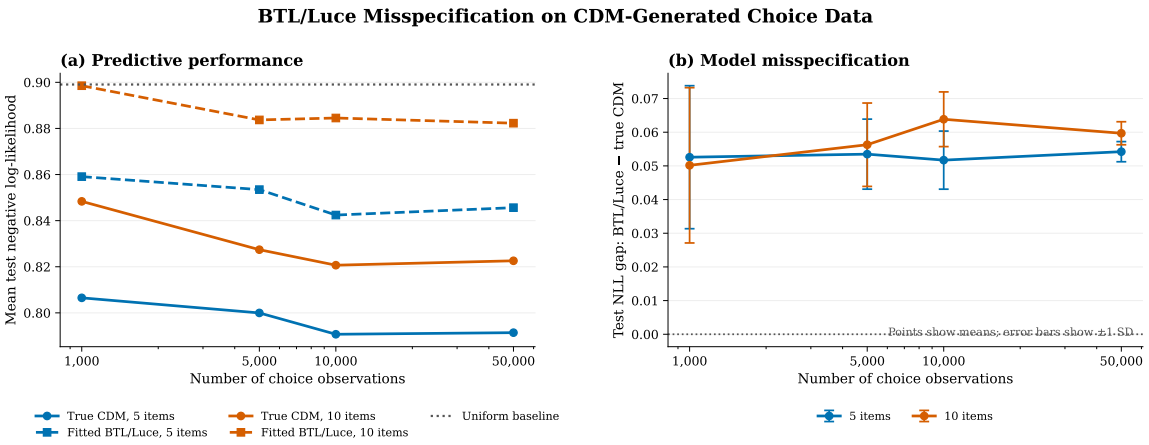


Figure 5.4: Predictive performance and test misspecification gap for BTL/Luce models fitted to CDM-generated choice data as the number of observations increases. Points represent means over ten independent runs, and error bars represent one standard deviation.

Table 5.5: Effect of CDM interaction strength on BTL/Luce model misspecification.

Items	Scale	True CDM NLL	BTL/Luce NLL	Gap	Gap SD
5	0.0000	0.8949	0.8949	0.0000	0.0001
5	0.2500	0.8644	0.8793	0.0148	0.0014
5	0.5000	0.7914	0.8456	0.0542	0.0030
5	0.7500	0.7056	0.8091	0.1035	0.0056
5	1.0000	0.6237	0.7773	0.1536	0.0063
10	0.0000	0.8949	0.8949	0.0001	0.0001
10	0.2500	0.8734	0.8909	0.0175	0.0017
10	0.5000	0.8226	0.8823	0.0597	0.0034
10	0.7500	0.7559	0.8719	0.1160	0.0041
10	1.0000	0.6889	0.8617	0.1728	0.0052

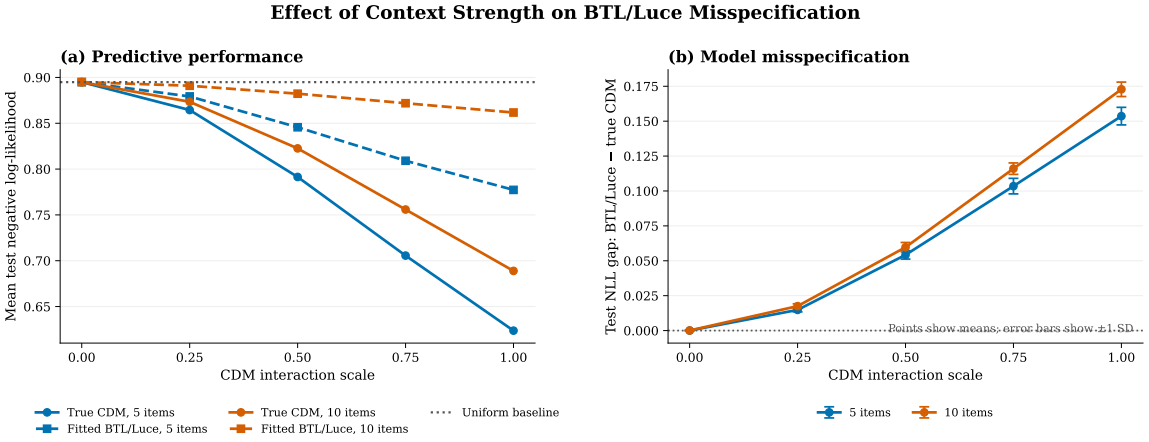


Figure 5.5: Effect of CDM interaction strength on the predictive performance and test misspecification gap of the fitted BTL/Luce model. Points represent means over ten independent runs, and error bars represent one standard deviation.

Appendix A

Additional Derivations

Appendix B

Reproducibility Details

Bibliography

[1] Arjun Seshadri, Alexander Peysakhovich, and Johan Ugander. Discovering context effects from raw choice data. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 5660–5669. PMLR, 2019.